

Quality Optimization of Live Streaming Services over HTTP with Reinforcement Learning

Farzad Tashtarian, Reyhane Falanji, Abdelhak Bentaleb, Alireza Erfanian, Peyman S. Mashhadi, Christian Timmerer, Hermann Hellwagner, and Roger Zimmermann

Christian Doppler laboratory ATHENA | Klagenfurt University | Austria
farzad.tashtarian@aau.at | <https://athena.itec.aau.at/>

Agenda

- Introduction
- Proposed approach
- Results
- Conclusion and future work
- Q&A

Introduction

- Recent years have seen tremendous growth in HTTP adaptive live video traffic over the Internet.
- HTTP Adaptive Streaming (HAS) has become the de-facto solution allowing Over-The-Top(OTT) services to deliver an acceptable Quality of Experience (QoE)
- Dynamic network conditions and diverse request patterns
 - A large overhead and significant increase in time complexity



HAS Player



HAS Player



HAS Player



CDN Server



CDN Server



CDN Server



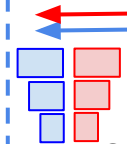
Live Source



ABR Encoder



Origin Server



Video Distribution Network

Video Contribution



HAS Player



HAS Player



HAS Player



CDN Server



CDN Server



CDN Server



Internet



Live Source



ABR Encoder



Origin Server

Video Distribution Network

Video Contribution



HAS Player



HAS Player



HAS Player

HTTP Request
for blue segment

HTTP Request
for red segment



CDN Server



CDN Server



CDN Server



Internet



Live
Source



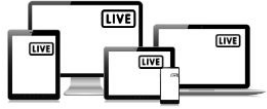
ABR
Encoder



Origin Server

Video Distribution Network

Video Contribution



HAS Player



HAS Player



HAS Player

HTTP Response
for blue segment



HTTP Response
for red segment



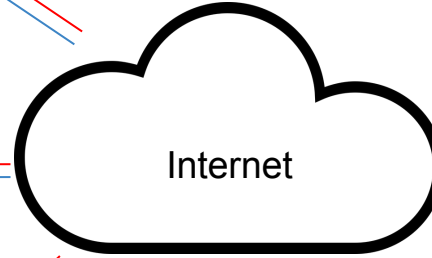
CDN Server



CDN Server



CDN Server



Internet



Live
Source



ABR
Encoder



Origin Server

Video Distribution Network

Video Contribution



HAS Player

HTTP Response
for blue segment

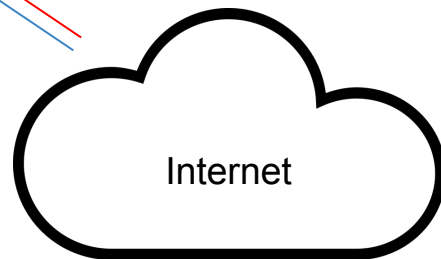


HTTP Response
for red segment



CDN Server

Video Distribution Network



Internet



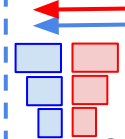
Live
Source



ABR
Encoder



Origin Server



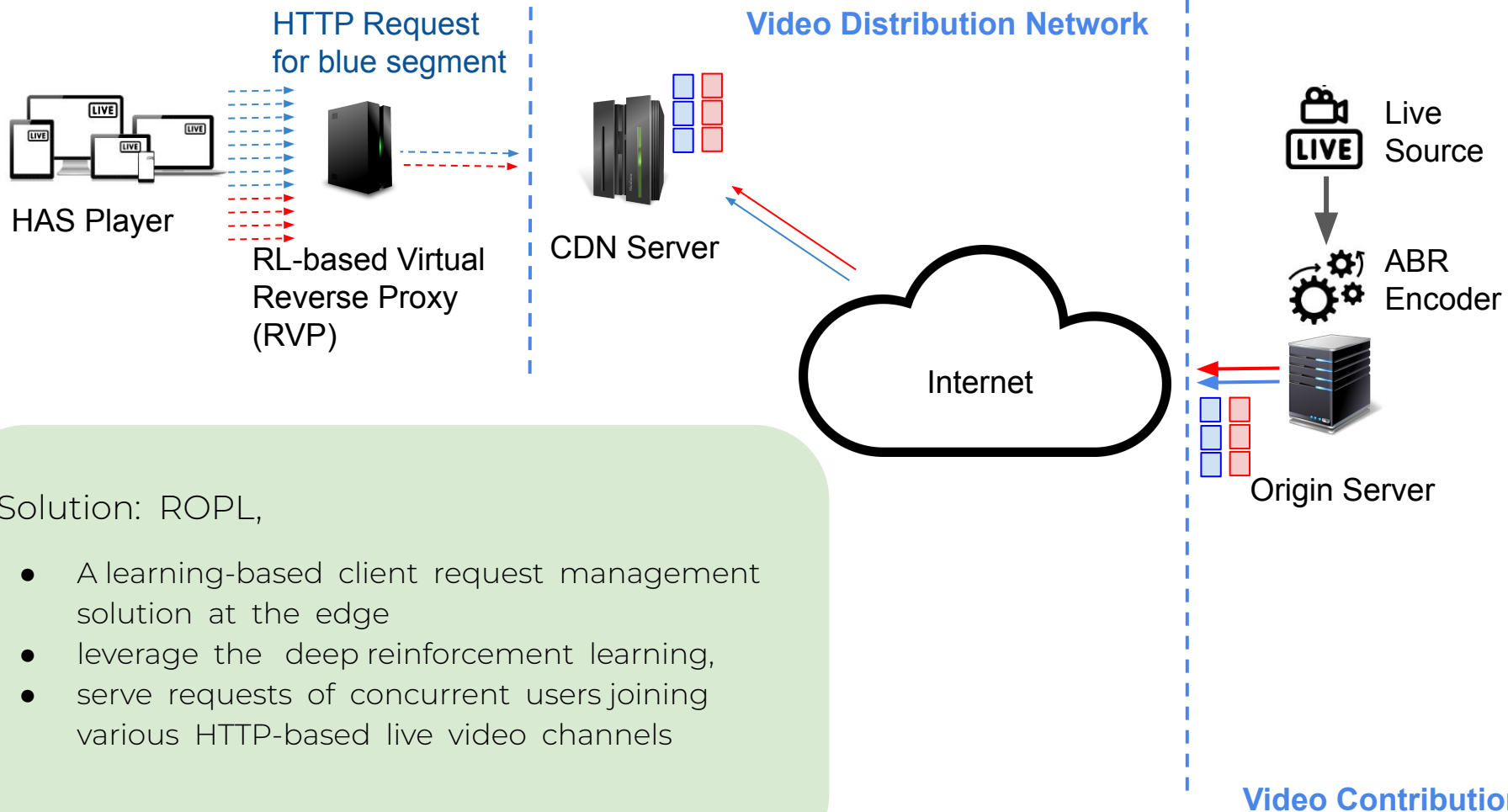
How to increase clients' QoE by considering :

- 1- Network Bandwidth between users and CDN server
- 2- Number of requests for a similar channel and quality
- 3- Different serving methods:
 - Fetch from CDN/origin server
 - Transcoding from higher quality to lower one
 - Serving with lower quality

Video Contribution

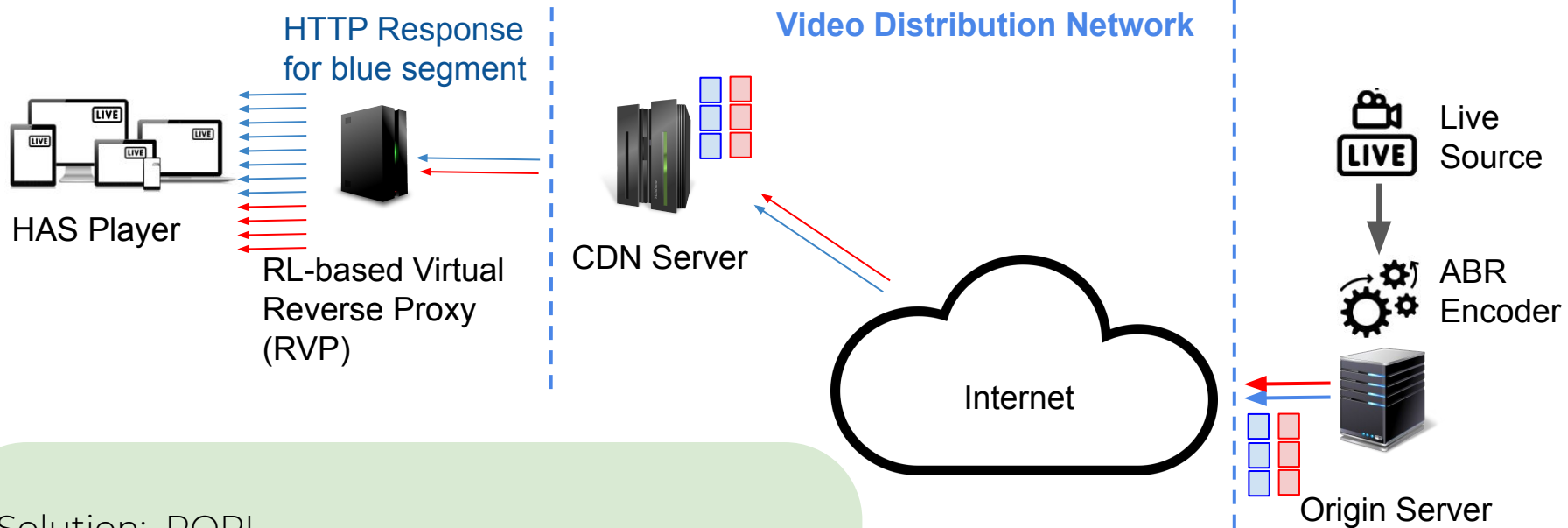
Problem Definition

How do we process and serve numerous clients' requests, demanding **different live channels** with **various bitrates**, at the network edge to elevate the clients' **perceived QoE** in a network with adverse bandwidth conditions?



Solution: ROPL,

- A learning-based client request management solution at the edge
- leverage the deep reinforcement learning,
- serve requests of concurrent users joining various HTTP-based live video channels

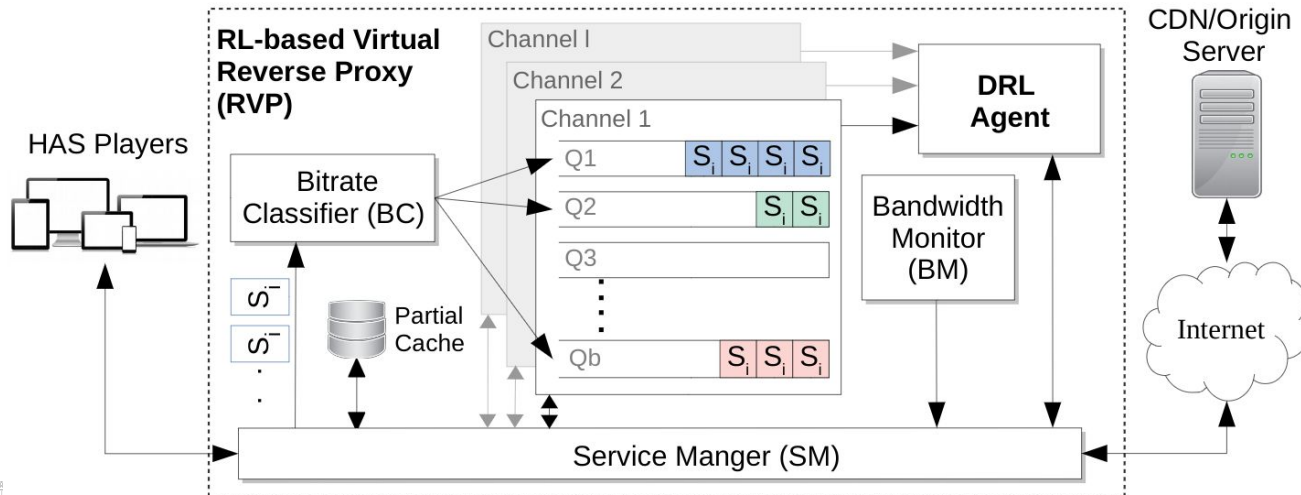


Solution: ROPL,

- A learning-based client request management solution at the edge
- leverage the deep reinforcement learning,
- serve requests of concurrent users joining various HTTP-based live video channels

ROPL Architecture

- The RVP consists of the following modules:
 - service manager(SM),
 - bitrate classifier (BC),
 - bandwidth monitor (BM),
 - partial cache,
 - **deep reinforcement learning (DRL)agent.**

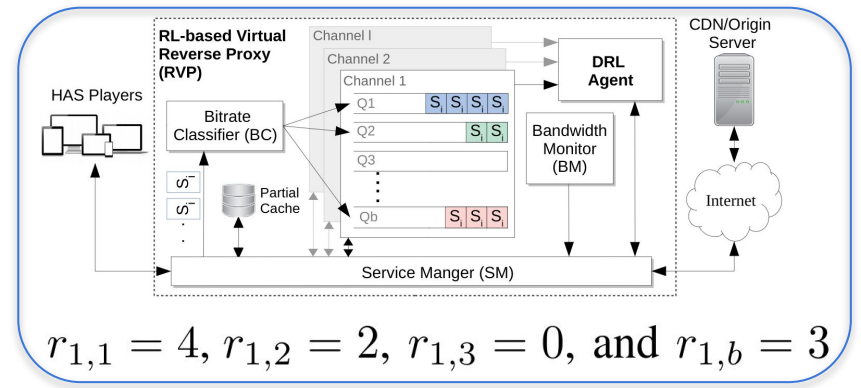


Deep Reinforcement Learning (DRL) Agent

- Proximal Policy Optimization (PPO)
- Policy gradient-based algorithm with an actor-critic approach
- Markov Decision Process (MDP) model
- The translation of our scenario into an MDP model is as follows:
 - State Space
 - Action Space
 - Policy
 - Reward Function

DRL Agent- State Space

Observation Space



$$O_{\tau} = (r_{1,1}, r_{1,2}, \dots, r_{1,b}, r_{2,1}, \dots, r_{l,b}, \omega)$$

$r_{i,j} \geq 0$ indicates the number of the same requests for bitrate j (considering an ascending order) of the live channel i

ω is the available bandwidth given from the SM module

DRL Agent- Action Space

$\mathcal{A}$ is a set of actions that can be performed by the DRL agent

$a_{i,j} \in \mathcal{A}$ Indicates the selected action for aggregated requests in the j th queue of the i th live channel $r_{i,j} \geq 0$ number

$$A_{\tau} = (a_{1,1}, a_{1,2}, \dots, a_{1,b}, a_{2,1}, \dots, a_{l,b})$$

A set of selected actions by the DRL agent at time step τ

DRL Agent- Action Space ...

ACT1: fetching the requested segment s with bitrate j directly from the remote CDN/origin server;

ACT2: serving segments with bitrate j^* demanded by request i^* in the same time step, where $j^* < j$ and action of i^* is ACT#1

ACT3: serving by transcoding segment s from a higher bitrate j^* demanded by request i^* where ACT#1 is selected for i^*

ACT4: do nothing

DRL Agent- Policy

- Policy refers to the logic based on which the agent takes action according to a particular observation.
- The policy **is optimized by learning** from the environment and updates based on reward signals.
- A desired convergence of policy happens when the agent learns to take the most rewarding action with the highest conditional probability, and a policy update does not change the current one.



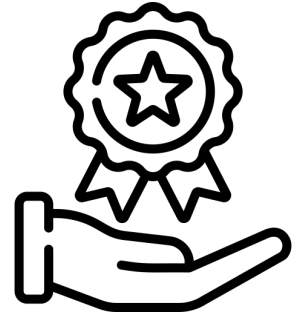
DRL Agent- Reward Function

- ❑ An undeniable role in the DRL
- ❑ Guides the agent towards the following purpose:

serving requests with the lowest cost and the maximum QoE

The proposed reward function consists of

- Serving cost
- Violation penalty



DRL Agent- Reward Function - Serving Cost

Serving cost C1-C4 for applying ACK1-ACK4 respectively.

Serving by fetch

$$C_1(i, j) = \alpha_1 \times s_{i,j}$$

A coefficient regarding the cost of bandwidth for applying ACT#1

Serving by transcoding

$$C_3(i, j) = \alpha_3 \times t_{i,\bar{j},j}$$

the required transcoding time

A coefficient regarding the cost of computational resources for applying ACT#3,

Serving by lower bitrates

$$C_2(i, j) = \alpha_2 \times r_{i,j} \times (\hat{j} - j)$$

by considering the cost for serving a requested bitrate with a lower one, coefficient α_2 is selected

$$C_4(i, j) = 0$$

DRL Agent- Reward Function - Violation Penalty

For every action set in a timestep, penalty signals are encountered as many as the number of violations according to the following constraints:

- **CON#1: empty request**
 - whenever any action other than ACT#4 is taken for an empty request
- **CON#2: bandwidth**
 - when a fetch action (ACT#1) is taken, but the bandwidth limit is violated.
- **CON#3: existing of lower bitrate**
 - when the agent decides to serve requests in a queue by a lower bitrate, however ACT#1 has not been taken for a queue that contains request(s) with similar segment number but lower bitrate;
- **CON#4: existing of higher bitrate**
 - when a queue is supposed to be served by transcoding from a higher bitrate version of the same segment, but no queue with a higher bitrate request has performed a successful fetch action (ACT#1);
- **CON#5: non-empty queue**
 - a “do nothing” decision for a non-empty queue.

DRL Agent- Reward Function

sum of actions' cost and violation in time step τ

sum of violation in time step τ

$$\mathcal{C}_\tau = (\mathbf{M}_c - \mathbf{C}_\tau^*)/\mathbf{M}_c, \text{ and } \mathcal{P}_\tau = (\mathbf{M}_p - \mathbf{V}_\tau^*)/\mathbf{M}_p$$

the normalized total cost in time step τ

the normalized total penalty in time step τ

$$Reward_\tau = \beta \times \mathcal{C}_\tau + (1 - \beta) \times \mathcal{P}_\tau, \text{ where } 0 \leq \beta \leq 1$$

Heuristic Post-Processing Algorithm

Prevent applying invalid actions on requests

Algorithm 1: Post-Processing Algorithm

```
1 for  $a_{i,j} \in I_\tau$ :  
2   if  $r_{i,j} = 0$  then:  $a_{i,j} \leftarrow \text{ACT\#4}$   
3   elif  $\text{len}[q \text{ for } q > j \text{ if } a_{i,q} == \text{ACT\#1} \& a_{i,q} \notin I_\tau] > 0$  then:  
4      $a_{i,j} \leftarrow \text{ACT\#3}$   
5   elif  $\text{len}[q \text{ for } q < j \text{ if } a_{i,q} == \text{ACT\#1} \& a_{i,q} \notin I_\tau] > 0$  then:  
6      $a_{i,j} \leftarrow \text{ACT\#2}$   
7   elif  $s_{i,j} \leq \omega^*$  then:  $a_{i,j} \leftarrow \text{ACT\#1}$   
8   elif  $s_{i,j^*} \leq \omega^*$  then:  $a_{i,j^*} \leftarrow \text{ACT\#1}$   
9   and for all requests of channel  $i$  with  $r_{i,j} > 0$  set ACT#2  
10  else: for all requests belonging to channel  $i$  set ACT#4
```

Performance Evaluation

	S.1	S.2	S.3	S.4	S.5	S.6
Channels	1	5	10	1	5	10
Players	50	250	500	50	250	500
BW [Mbps]	1–4.2	5–21	10–42	2–5	10–30	20–60

We conduct the performance evaluation in two modes:

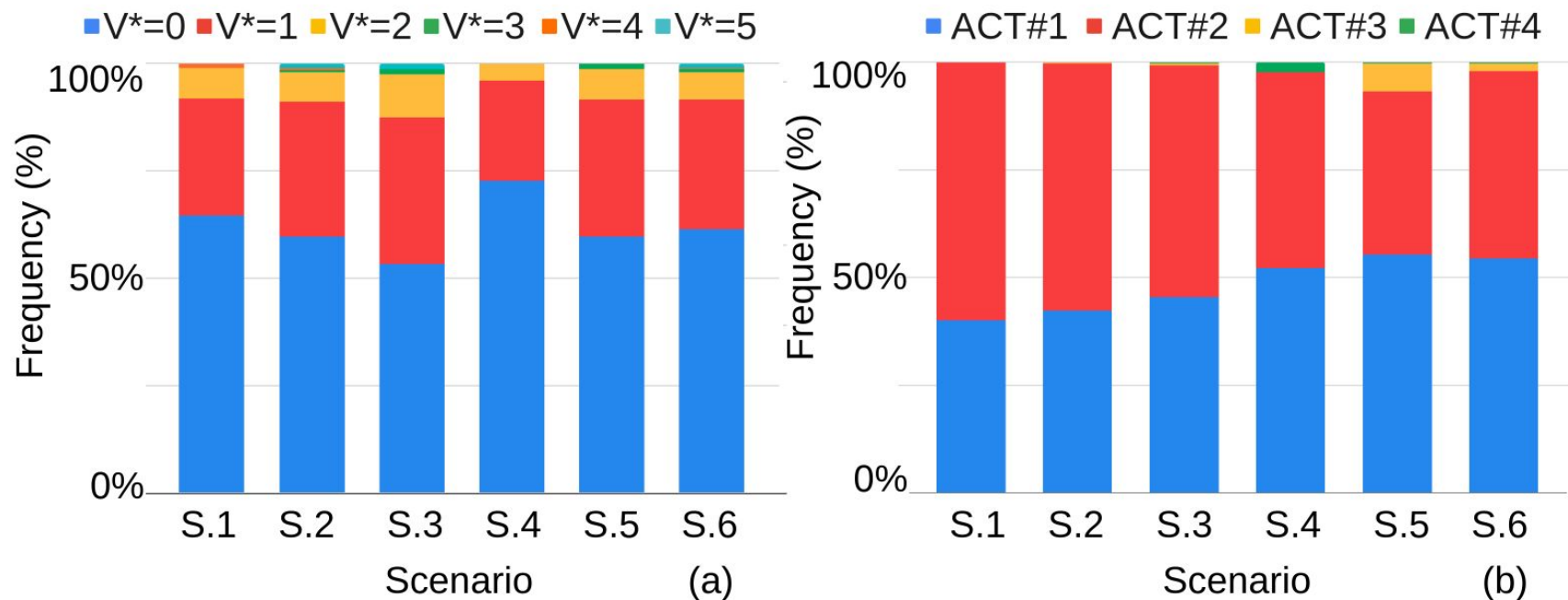
- Simulation scenarios
 - six scenarios for different numbers of live channels, players, and various amounts of bandwidth
- Real-world scenarios
 - RVP in Python to serve requests received from the goDASH player
 - HTTP origin server on an AWS virtual machine located in the Frankfurt zone
 - CH1(Big Buck Bunny), CH2(Tears of Steel), CH3(Sintel)

CH1: {180p@0.25Mbps, 360p@1.1Mbps, 414p@1.82Mbps, 720p@3.0Mbps, 1080p@3.89Mbps}

CH2: {216p@0.39Mbps, 360p@1.1Mbps, 414p@1.82Mbps, 720p@2.38Mbps, 1080p@3.89Mbps}

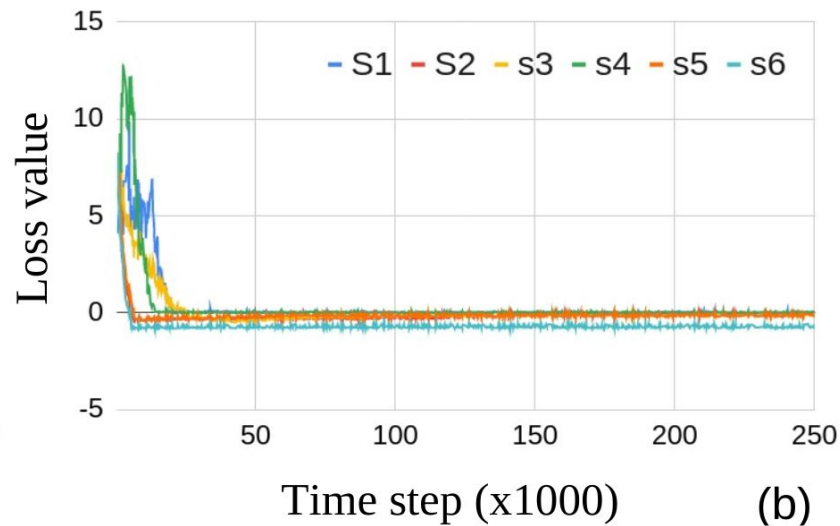
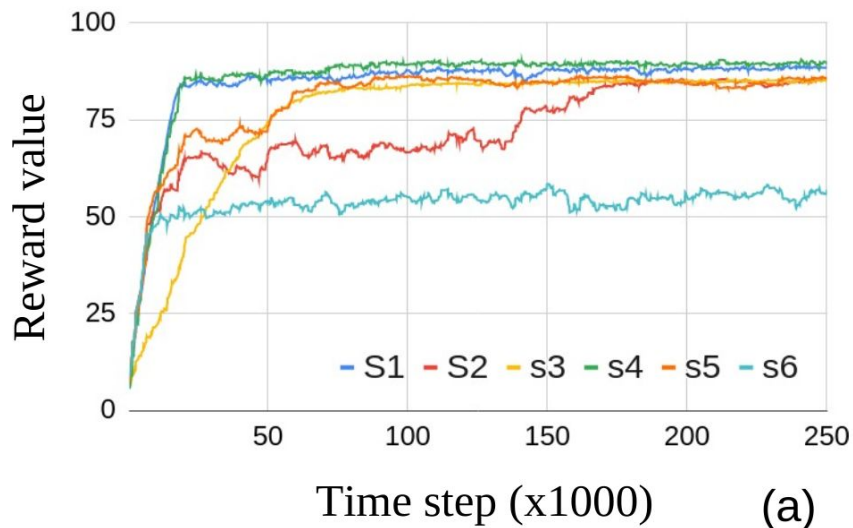
CH3: {288p@0.57Mbps, 414p@1.82Mbps, 720p@2.38Mbps, 1080p@3.89Mbps, 1080p@4.3Mbps}

Simulation Experiments



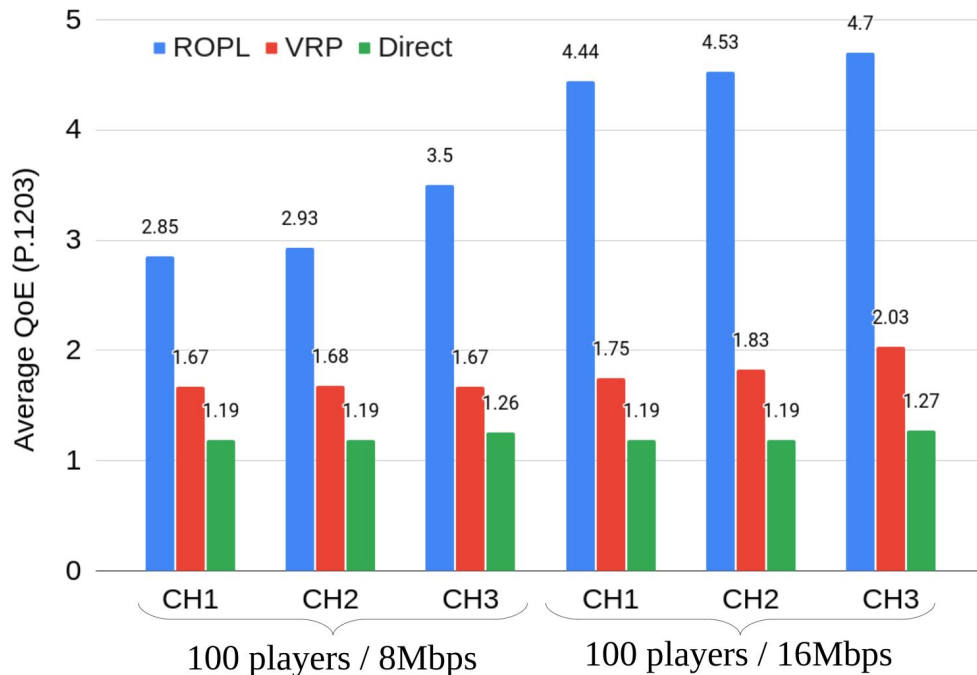
Frequency of penalties (a) and taken actions (b) for different scenarios using ROPL model in simulation

Simulation Experiments ...



Resulted reward (a) and loss (b) using ROPL model in simulations

Real-world Experiment



Achieved average QoE for each channel clients and different total bandwidth configurations using ROPL in real-world experiments.

Conclusion and Future Work

- We proposed ROPL,
 - a reinforcement learning approach
 - optimizing HTTP-based live streaming
 - an edge based mechanism
 - trace-driven simulations and a real-world setup
 - outperforms existing heuristic-based approaches in terms of QoE, with a factor up to 3.7×
- Extend ROPL to support multiple RVPs

Quality Optimization of Live Streaming Services over HTTP with Reinforcement Learning

Farzad Tashtarian, Reyhane Falanji, Abdelhak Bentaleb, Alireza Erfanian, Peyman S. Mashhadi, Christian Timmerer, Hermann Hellwagner, and Roger Zimmermann

Christian Doppler laboratory ATHENA | Klagenfurt University | Austria
farzad.tashtarian@aau.at | <https://athena.itec.aau.at/>